



Introdução ao Node.js

Runtime, Módulos e Servidor HTTP

Agenda da Aula



01 Node.js como Runtime

V8, libuv, arquitetura e event loop



02 Módulos Internos (Core)

os, path, fs e http



03 Servidor HTTP do Zero

Criação, roteamento e respostas

O que é Node.js?

Ambiente de execução (runtime) que permite rodar JavaScript fora do navegador, diretamente no servidor.



Criado em 2009

Por Ryan Dahl, para resolver problemas de I/O em servidores web



Motor V8

Usa o mesmo motor JavaScript do Google Chrome



Multiplataforma

Roda em Windows, macOS e Linux sem alterações

JavaScript: Navegador vs Servidor



Navegador (Browser)

- Acesso ao DOM (document)
- Objeto window e alert()
- Eventos de clique e teclado
- Manipulação de interface
- Fetch API / XMLHttpRequest
- LocalStorage / SessionStorage

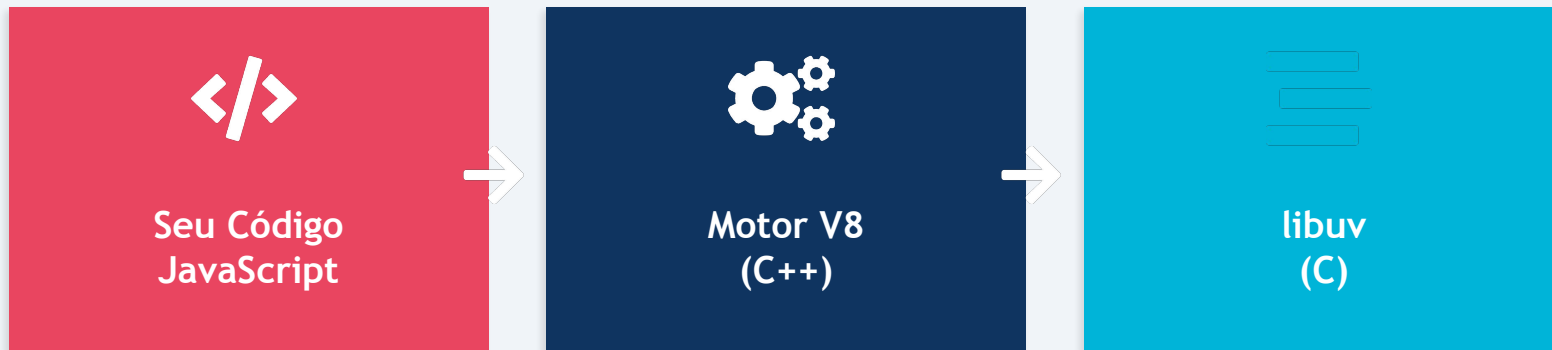


Node.js (Servidor)

- Acesso ao sistema de arquivos (fs)
- Criação de servidores HTTP
- Processos e threads do SO
- Variáveis de ambiente
- Streams e buffers binários
- Conexões com bancos de dados

Mesma linguagem, contextos completamente diferentes!

Arquitetura do Node.js



Você escreve código JS utilizando módulos e APIs do Node.js

Compila e executa o JavaScript em código de máquina nativo

Gerencia I/O assíncrono, event loop, threads e rede

```
Node.js APIs:  http  |  fs   |  path  |  os   |  events  |  stream
                |  crypto  |  url
```

Event Loop e I/O Não-Bloqueante

Node.js é single-threaded, mas altamente eficiente graças ao seu modelo assíncrono e event loop.



Analogia: O Garçom

Imagine um garçom em um restaurante:

- Ele anota o pedido da mesa 1
- Não espera o prato ficar pronto
- Vai atender a mesa 2, mesa 3...
- Quando a cozinha avisa, ele entrega

O garçom é a thread principal.

A cozinha é o I/O assíncrono.

Ciclo do Event Loop

1

Recebe requisição

2

Delega I/O para libuv

3

Continua processando

4

Callback executado

Síncrono vs Assíncrono na Prática

Bloqueante (Síncrono)

```
const dados =  
  fs.readFileSync('arquivo.txt');  
console.log(dados);  
console.log('Próxima tarefa');
```

A execução PARA até o arquivo ser lido completamente.

Nenhuma outra requisição é atendida nesse tempo.

Não-Bloqueante (Assíncrono)

```
fs.readFile('arquivo.txt',  
  (err, dados) => {  
    console.log(dados);  
  });  
console.log('Próxima tarefa');
```

A execução CONTINUA imediatamente.

O callback roda quando o I/O terminar!

Quem usa Node.js e por quê?

Netflix

Reduziu tempo de startup em 70%

LinkedIn

Migrou de Ruby: 27 servidores para 3

PayPal

Dobrou as requisições por segundo

Uber

Processa milhões de viagens em tempo real

Por que essas empresas escolheram Node.js?

- Alta performance em operações de I/O
- JavaScript no frontend e backend (full-stack)
- Ecossistema massivo de pacotes e comunidade ativa

Módulos Internos do Node.js

Core Modules: os, path, fs, http

CommonJS vs ES Modules

CommonJS (CJS)

```
// Exportar  
module.exports = minhaFuncao;  
  
// Importar  
const fn = require('./arquivo');
```

Padrão original do Node.js
Carregamento síncrono

ES Modules (ESM)

```
// Exportar  
export default minhaFuncao;  
  
// Importar  
import fn from './arquivo.js';
```

Padrão moderno (ECMAScript)
"type": "module" no package.json

Nesta aula usaremos CommonJS (require), pois é o padrão mais comum em projetos Node.js existentes.

Core Modules: os e path

os

Informações do Sistema Operacional

```
const os = require('os');  
  
os.platform()    // 'linux'  
os.cpus()        // info CPUs  
os.totalmem()    // RAM total  
os.freemem()     // RAM livre  
os.homedir()     // '/home/user'  
os.hostname()    // nome da máquina
```

path

Manipulação de Caminhos

```
const path = require('path');  
  
path.join('pasta', 'arquivo.js')  
// 'pasta/arquivo.js'  
  
path.resolve('arquivo.js')  
// caminho absoluto  
  
path.extname('foto.png')  
// '.png'
```

Core Module: fs (File System)

Ler Arquivo

```
const fs = require('fs');
fs.readFile('dados.txt', 'utf8',
  (err, conteudo) => {
    console.log(conteudo);
  });
```

Escrever Arquivo

```
const texto = 'Olá, Node.js!';
fs.writeFile('saida.txt', texto,
  (err) => {
    if (err) throw err;
    console.log('Salvo!');
  });
```

Outras operações úteis do fs

```
fs.existsSync('arquivo.txt') // Verifica se existe
fs.mkdirSync('nova-pasta')    // Cria diretório
fs.readdirSync('.')            // Lista arquivos
fs.unlinkSync('arquivo.txt')  // Remove arquivo
```

Core Module: http



O módulo http permite criar servidores web completos sem nenhuma dependência externa. É a base sobre a qual frameworks como Express.js são construídos.

O que o módulo http permite fazer:



Criar servidores que escutam requisições em uma porta



Processar requisições HTTP (GET, POST, PUT, DELETE)



Definir rotas e direcionar para diferentes respostas

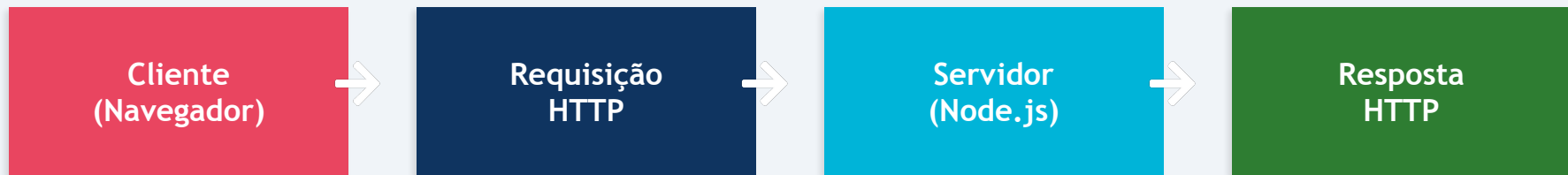


Servir páginas HTML, arquivos JSON ou qualquer conteúdo

Servidor HTTP do Zero

Criando, roteando e respondendo requisições

Como a Web Funciona (Revisão Rápida)



Métodos HTTP

- GET — buscar dados/páginas
- POST — enviar/criar dados
- PUT — atualizar dados existentes
- DELETE — remover dados

Status Codes

- 200 — OK (sucesso)
- 201 — Created (criado)
- 404 — Not Found (não encontrado)
- 500 — Internal Server Error

Criando um Servidor HTTP

server.js

```
const http = require('http');

const servidor = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Olá, Mundo!');
});

servidor.listen(3000, () => {
  console.log('Rodando na porta 3000');
});
```

Passo a passo:

1. Importar o módulo http
2. Criar servidor com createServer e definir o callback
3. Configurar a resposta (status, headers, corpo)
4. Iniciar escuta na porta com listen()

Anatomia de req e res

req (Requisição — entrada)

req.method

'GET', 'POST', 'PUT', 'DELETE'

req.url

'/api/usuarios', '/sobre'

req.headers

Objeto com todos os cabeçalhos

res (Resposta — saída)

res.statusCode = 200

Define o código de status HTTP

res.setHeader('Content-Type', ...)

Define cabeçalhos da resposta

res.write() / res.end()

Envia corpo e finaliza resposta

Roteamento Básico e Content-Type

rotas.js

```
const servidor = http.createServer((req, res) => {  
  if (req.url === '/') {  
    res.setHeader('Content-Type', 'text/html');  
    res.end('<h1>Página Inicial</h1>');  
  } else if (req.url === '/api') {  
    res.setHeader('Content-Type', 'application/json');  
    res.end(JSON.stringify({ msg: 'Olá API' }));  
  } else {  
    res.statusCode = 404;  
    res.end('Página não encontrada');  
  }  
});
```

O Content-Type diz ao navegador como interpretar a resposta: text/html, application/json, text/plain, etc.

Do HTTP Nativo ao Express.js

Limitações do http nativo

- Roteamento manual com if/else
- Sem suporte nativo a middlewares
- Parsing de body manual
- Servir arquivos estáticos é trabalhoso
- Muito código para tarefas simples

Express.js (próxima aula)

- Roteamento elegante e organizado
- Sistema de middlewares poderoso
- Parsing automático de JSON
- Servir arquivos estáticos facilmente
- Framework mais popular do Node.js

Mas primeiro, é fundamental entender como tudo funciona por baixo dos panos!

Resumo da Aula

01

Runtime

Node.js usa V8 + libuv para executar JS no servidor com I/O não-bloqueante e event loop

02

Core Modules

os (sistema), path (caminhos), fs (arquivos) e http (servidores) são módulos nativos prontos para uso

03

Servidor HTTP

Com poucas linhas criamos um servidor, definimos rotas e respondemos com HTML ou JSON